CSE 4165 · Web Programming

Final Exam Cheat Sheet

SIDE 1 · JavaScript + PHP logic

HOW TO USE THIS SHEET 1 Read the question and underline the key words. 2 Find them in the DECODER below. 3 Copy the block whose USE WHEN tag matches, rename the variables. 4 Test EVERY sample row from the paper. Colours: JavaScript PHP MySQL exam rules

DECODER words in the question → what to write

JAVASCRIPT

- **"keep track · so far · running total"** → global `let total = 0;`
- **"average"** → `total / count`
- **"more than N tries without …"** → `count > N && !reached`
- **"+X points if present"** → `/[A-Z]/.test(p)`
- **"levels · ranges 0-30, 31-50…"** → if / else if
- **"show feedback in the label"** → `innerText` / `innerHTML`

PHP

- **"complete · no partial · minimum number"** → `ceil()`
- **"left over · empty · remaining"** → `units*cap - people`
- **"classify · category · status"** → if / elseif ladder
- **"above / below / exactly target"** → `$total - $target`
- **"best two of three"** → `a+b+c - min(a,b,c)`
- **"pass / fail"** → `? :` short if

MySQL

- **"how many … in each"** → `COUNT(*) GROUP BY`
- **"only groups with more than"** → `HAVING`
- **"increase by 10% · only if"** → `SET x = x*1.10`
- **"largest first · sorted"** → `ORDER BY … DESC`
- **"label … otherwise …"** → `CASE WHEN … END`
- **"above its own group's average"** → subquery

JAVASCRIPT

Q1 type · type something → click a button → live feedback

1 variables **outside** the function (they remember) 2 read the boxes 3 check not empty / NaN 4 update total / count / list 5 pick the message (biggest limit first) 6 print to the label

USE WHEN a tracker: calories, study hours, anything "so far"

```
<!-- index.html : every id here is used by script.js -->
<input type="text"   id="name">
<input type="number" id="amount" step="any">
<button onclick="go()">Add</button>   <!-- runs go() -->
<p id="out"></p>                      <!-- result label -->
<script src="script.js"></script>     <!-- keep LAST -->
```

```
// script.js
const GOAL = 100;                 // 1) fixed numbers of the question
let total = 0, count = 0, names = [];   // 2) memory: OUTSIDE
function go() {
  const out = document.getElementById("out");
  const name = document.getElementById("name").value.trim();
  const x = parseFloat(document.getElementById("amount").value);
  if (name === "" || isNaN(x)) {   // 3) check the input
    out.innerText = "Enter valid input";
    return;                        //    stop here
  }
  total += x;  count++;  names.push(name);   // 4) update memory
  const avg = total / count;
  let msg;                         // 5) message: BIGGEST limit first
  if (total >= 100)      msg = "Goal reached!";
  else if (total >= 50)  msg = "Halfway there!";
  else                   msg = "Keep going!";
  if (count > 7 && total < GOAL) // too many entries: warning
    msg = "Increase your effort!";
  out.innerHTML =                  // 6) print
    `Total: ${total}<br>Average: ${avg}<br>` +
    `Items: [${names.join(", ")}]<br>${msg}`;
}
```

TIP **innerHTML** understands
 (new line); **innerText** shows plain text. Button does nothing? Press **F12 → Console**: the red error names the typo. **To adapt:** change the ids, GOAL, limits and messages — keep the 6 steps.

USE WHEN a points checklist: password strength, scores

```
function score(p) {                  // p = the text typed
  let s = 0;
  if (p.length >= 6) s += Math.floor((p.length - 6) / 2) * 10;
  if (/[A-Z]/.test(p))      s += 15;  // has a capital letter
  if (/[a-z]/.test(p))      s += 15;  // has a small letter
  if (/[0-9]/.test(p))      s += 20;  // has a digit
  if (/[!@#$%^&*]/.test(p)) s += 25;  // has a symbol
  return s;
}
```

TRAP Unclear wording like "+10 per 2 characters (min 6)"? Work the sample out **by hand both ways**; keep the reading that matches (here floor((len−6)/2)×10). Messages must match **letter for letter**, incl. **!**

USE WHEN "more than N attempts without reaching …" warning

```
let attempts = 0, reached = false;   // outside the function
// ---- inside the function, on every click:
attempts++;
if (s >= 71) reached = true;         // remember success for good
if (attempts > 8 && !reached)        // > 8 tries and no success
  msg = "Need practice! Tips: use 8+ characters, mix upper and " +
        "lower case, add digits and symbols.";
```

USE WHEN many readings, averages, risk formulas (arrays)

```
let hr = [], sp = [];              // one array per input box
hr.push(h);  sp.push(s);           // store every new reading
const avg = a => a.reduce((x, y) => x + y, 0) / a.length;
avg(hr);                           // average of ALL so far
Math.abs(h - 80);                  // distance, never negative
+avg(sp).toFixed(2);               // 90.6666 -> 90.67   97 -> 97
Math.max(...hr);                   // biggest (min = smallest)
```

PHP

Q2 type · form → calculate → print the answer

1 form.html: every box has a **name=** 2 result.php reads **$_POST** 3 convert (int) / (float) 4 calculate inside a function 5 echo it exactly like the sample

USE WHEN any form + function question (start here)

```
<!-- form.html : name="..." is what PHP reads -->
<form method="post" action="result.php">
  A: <input type="number" name="a" required><br>
  B: <input type="number" name="b" required><br>
  C: <input type="number" name="c" required><br>
  <input type="submit" value="Calculate">
</form>
```

```
<?php  // result.php
function calc($a, $b, $c) {      // the question's function
  $total = $a * $b;              // ...your formulas here...
  $left  = $total - $c;
  return [$total, $left];        // give back several values
}
$a = (int) $_POST["a"];          // read the box named "a"
$b = (int) $_POST["b"];          // (int) whole, (float) decimal
$c = (int) $_POST["c"];
[$total, $left] = calc($a, $b, $c);   // catch the values
echo "Total: $total<br>";        // <br> = new line on the page
echo "Left: $left<br>";
echo "Money: " . number_format($total * 120);  // 36,000
?>
```

TIP Open PHP through **http://localhost/web_final/…**, never by double-click (you would see raw code). Blank white page = a typo: look for a missing **;** or a missing quote. All variables start with **$**.

USE WHEN "complete / cannot book partial / how many are needed"

```
$units = (int) ceil($people / $capacity);  // round UP: 3.1 -> 4
$empty = $units * $capacity - $people;     // seats left over
// $people % $capacity = remainder.  floor() rounds DOWN.
```

USE WHEN "classify / category / level / status" from ranges

```
if ($t >= 120)     $cat = "Excellent";  // BIGGEST first
elseif ($t >= 90)  $cat = "Good";
elseif ($t >= 60)  $cat = "Average";
else               $cat = "Poor";       // all the rest
```

USE WHEN "above / below / exactly at the target"

```
$d = $total - $target;       // + above, - below
if ($d > 0)      $res = "Above target by $d";
elseif ($d < 0)  $res = "Below target by " . abs($d);
else             $res = "Target met exactly (0 difference)";
```

USE WHEN percent · best two of three · pass / fail

```
$pct  = $unused / $carried * 100;    // percent of something
$best = $a + $b + $c - min($a, $b, $c);  // best TWO of three
$avg  = $best / 2;                   // their average
$status = ($total > 54) ? "Passed" : "Failed";  // short if/else
// many values: rsort($arr); $arr[0] + $arr[1] = best two
```

USE WHEN a list of values · total, biggest, smallest · repeat

```
$marks = [12, 15, 14];                  // a list of values
foreach ($marks as $m) echo "$m ";      // for each item
echo count($marks), " ", array_sum($marks), " ",
     max($marks), " ", min($marks);     // size,total,max,min
for ($i = 1; $i <= 5; $i++) echo "$i<br>";   // repeat 5 times
```

TRAP A cost in the story (venue 15,000 · rental 25,000) can be a **distractor**: compute what the **sample table** shows (e.g. 70 empty × 120 = 8,400). "More than 54" means **>**, "54 or more" means **>=**. If the text and the sample disagree, follow the text and add a comment.

CSE 4165 · Web Programming

Final Exam Cheat Sheet

SIDE 2 · MySQL + exam day

MySQL · PART 1 Q3 type · build the table, add the rows, run the queries

HINT **Marks: 50% database.sql** (CREATE + INSERT) **+ 50% the queries.** Hand in the .sql file **and** the PHP page. Do the .sql first — it is the easy half.

1 write **database.sql** (table + all rows) **2** phpMyAdmin → **Import** → Go **3** try each query in the SQL tab **4** copy them into **q3.php** **5** open q3.php in the browser

USE WHEN **always first: creates the database, the table and every row**

```
-- database.sql
CREATE DATABASE IF NOT EXISTS db_name;    -- name from question
USE db_name;
DROP TABLE IF EXISTS table_name;          -- lets you re-import
CREATE TABLE table_name (
  id         INT PRIMARY KEY,          -- whole number
  name       VARCHAR(50),              -- text
  price      DECIMAL(10,2),            -- money / decimals
  rating     DECIMAL(3,1),             -- 4.5
  grade      CHAR(1),                  -- one letter
  added_on   DATE                      -- '2026-10-15'
);
INSERT INTO table_name
  (id, name, price, rating, grade, added_on) VALUES
  (1, 'Pen',  12.50, 4.5, 'A', '2026-10-15'),
  (2, 'Book', 80.00, 4.1, 'B', '2026-10-16');
-- now write every task query here, each with a -- comment
```

TYPES `INT` whole · `DECIMAL(10,2)` money · `DECIMAL(3,1)` rating 4.5 · `VARCHAR(50)` text · `CHAR(1)` one letter · `DATE` dates

TIP Copy table and column names **exactly** as the question spells them and enter **every** row. Text and dates in **'single quotes'**, numbers without. An apostrophe inside text is doubled: `'Cox''s Bazar'`.

USE WHEN **the PHP page that runs the queries and prints the results**

```
<?php  // q3.php  (database.sql already imported)
$conn = new mysqli("localhost", "root", "", "db_name");
if ($conn->connect_error)
  die("Connection failed: " . $conn->connect_error);
// show(): runs any SELECT and prints a table
function show($conn, $title, $sql) {
  echo "<h4>$title</h4>";
  $r = $conn->query($sql);
  if (!$r) { echo "Error: " . $conn->error; return; }
  if ($r->num_rows > 0) {
    echo "<table border='1'><tr>";
    foreach ($r->fetch_fields() as $f)
      echo "<th>{$f->name}</th>";
    echo "</tr>";
    while ($row = $r->fetch_assoc()) {
      echo "<tr>";
      foreach ($row as $v) echo "<td>$v</td>";
      echo "</tr>";
    }
    echo "</table>";
  } else echo "0 results";
}
// run(): runs UPDATE / DELETE / INSERT
function run($conn, $title, $sql) {
  if ($conn->query($sql) === TRUE)
    echo "<p>$title: {$conn->affected_rows} row(s)</p>";
  else echo "<p>Error: " . $conn->error . "</p>";
}
// the tasks, IN THE ORDER LISTED
show($conn, "Task 1", "SELECT name, price FROM table_name");
run($conn, "Task 2",
  "UPDATE table_name SET grade='C' WHERE price < 20");
show($conn, "Check", "SELECT * FROM table_name");
$conn->close();
?>
```

USE WHEN **a task wants ONE number (net worth, total, count)**

```
$r = $conn->query("SELECT SUM(price * stock) AS total FROM t");
$row = $r->fetch_assoc();              // one row, one number
echo "Net worth: " . number_format($row["total"]);
```

XAMPP QUICK START how to run everything

1 XAMPP Control Panel → **Start** Apache + MySQL (both turn green) **2** all files live in **C:\xampp\htdocs\web_final** **3** PHP: open **http://localhost/web_final/q3.php** · JavaScript: double-click index.html **4** import: **http://localhost/phpmyadmin** → Import → Choose file → Go **5** test one query: click your database → **SQL** tab → paste → Go **6** table messed up? import database.sql again (it drops and rebuilds)

MySQL · QUERY COOKBOOK sentence → pattern (t = your table)

USE WHEN **show · filter · sort**

```
SELECT name, price FROM t;                -- chosen columns
SELECT * FROM t ORDER BY price DESC;      -- biggest first
SELECT * FROM t WHERE category = 'Book' -- text in 'quotes'
  AND price > 400;                        -- AND / OR
SELECT * FROM t WHERE name LIKE '%Beach%';   -- contains a word
SELECT * FROM t WHERE rating BETWEEN 4.0 AND 4.5;
SELECT * FROM t WHERE category IN ('A', 'B');
-- not equal: <>   at least: >=   two sorts: ORDER BY a DESC, b
```

USE WHEN **count · total · average per group · only some groups · highest**

```
SELECT category, COUNT(*) AS n            -- how many in each
FROM t GROUP BY category;
SELECT category, SUM(price) AS total,     -- total per group
       AVG(rating) AS avg_r               -- average per group
FROM t GROUP BY category
HAVING COUNT(*) > 1                       -- keep only some groups
ORDER BY total DESC;                      -- largest first
SELECT SUM(price * stock) FROM t;         -- whole table, no GROUP BY
SELECT * FROM t ORDER BY price DESC LIMIT 1;   -- the highest row
```

USE WHEN **"if … update …" · increase by 10% · only if the result …**

```
UPDATE t SET grade = 'C'                  -- change a value if ...
 WHERE salary < 40000 AND grade <> 'D';
UPDATE t SET price = price * 1.10         -- +10%  (-10% = * 0.90)
 WHERE price > 500;
UPDATE t SET salary = salary + 5000       -- bonus with a CAP:
 WHERE salary > 50000                     -- test the NEW value
   AND salary + 5000 <= 60000;            -- in WHERE as well
UPDATE t SET status = 'Grace', fee = 0  -- two columns at once
 WHERE status = 'Overdue' AND days < 7;
```

USE WHEN **show a label · compare with the group's average**

```
SELECT name, price,                       -- show a label
  CASE WHEN price > 100 THEN 'Top Seller'
       ELSE 'Regular Seller' END AS label
FROM t;
SELECT name, category,         -- above ITS OWN group's average
  CASE WHEN price > (SELECT AVG(price) FROM t t2
                     WHERE t2.category = t.category)
       THEN 'Top' ELSE 'Regular' END AS label
FROM t;
```

USE WHEN **delete · insert · add a column · distinct · round**

```
DELETE FROM t WHERE stock = 0;            -- ALWAYS with WHERE
INSERT INTO t (id, name) VALUES (7, 'New');
ALTER TABLE t ADD bonus DECIMAL(10,2) DEFAULT 0;
SELECT COUNT(DISTINCT category) FROM t; -- how many different
SELECT ROUND(AVG(price), 2) FROM t;       -- 2 decimals
SELECT CONCAT(first, ' ', last) FROM t; -- glue text together
```

TRAP **UPDATE / DELETE without WHERE changes EVERY row.** COUNT / SUM / AVG never go in WHERE — use **HAVING**. Run the tasks in the listed order: earlier updates change later results. A group with no rows (a grade nobody has) is missing from GROUP BY — print its 0 yourself.

HINT **Worked example — turning a sentence into SQL: for each** department = GROUP BY · **number of** employees = COUNT(*) · **largest first** = ORDER BY n DESC.

```
-- "per department, number of staff, largest first"
SELECT department, COUNT(*) AS n
FROM t GROUP BY department ORDER BY n DESC;
```

EXAM DAY the lecturer's rules in 20 seconds

8:45 at the lab: start **Apache + MySQL** in XAMPP. A port is busy? Kill the process on it (shown in class). Tell the invigilator **before** 9:00. **Work only inside web_final.** **10:55** zip **web_final** → upload to eLMS. **No code edits 10:55–11:00.** Late or emailed = 0 (AB). Know your eLMS password.
Allowed: this one printed, double-sided page. **No** phone, laptop, pendrive, calculator or notes. Sharing code = 0 for both students. No washroom 9:00–11:00.
Before you zip: every sample row tested · database.sql imports with no error · q3.php prints every task · messages spelled like the sample · everything is inside web_final.

WHEN IT BREAKS quick fixes

Button does nothing → F12 Console; the function name or an id does not match. **NaN** → empty box, or you forgot parseFloat. **Raw PHP code on screen** → open via http://localhost, not double-click. **Blank page** → missing ; or quote. **Unknown column** → spelling. **Duplicate entry** → same id twice. **Access denied** → user *root*, password empty ("").